Spring JDBC – Complete Notes with CRUD Operations

Spring JDBC is a module of the Spring Framework that simplifies database operations using JDBC. It provides the JdbcTemplate class, which reduces repetitive JDBC code and makes it easier to perform Create, Read, Update, and Delete (CRUD) operations.

We will develop a simple Student Management Application using Spring Boot, Spring JDBC, and MySQL.

1. What is Spring JDBC?

1. Spring JDBC is a database-access technology provided by the Spring Framework.
2. It is built on top of the standard Java JDBC API.
3. It simplifies database connection management, SQL execution, exception handling, and resource cleanup.
4. It provides the JdbcTemplate class for executing SQL queries.
5. Developers write SQL queries manually.
6. It supports CRUD operations, batch processing, and transaction management.
7. It works with relational databases such as MySQL, PostgreSQL, Oracle, and SQL Server.
8. It does not require Hibernate or JPA.
9. It integrates with Spring Boot's dependency injection and auto-configuration.
10. It converts JDBC exceptions into Spring's DataAccessException hierarchy.

2. JDBC vs Spring JDBC

Feature	Traditional JDBC	Spring JDBC
Database connection	Manually obtain connection	Managed through DataSource
SQL execution	Statement / PreparedStatement	JdbcTemplate
Resource management	Manual or try-with-resources	Automatically handled
Exception handling	SQLException	DataAccessException
ResultSet processing	Manual	RowMapper
Boilerplate code	More	Less
SQL queries	Manual	Manual
Transaction management	Manual or external	Supports Spring transactions

3. Spring JDBC Architecture

Request flow for a Spring Boot application using Spring JDBC

The controller receives a request, the service handles business logic, and the repository uses JdbcTemplate to interact with MySQL.

4. Important Spring JDBC Components

JdbcTemplate

JdbcTemplate is the central class of Spring JDBC.

It provides methods such as:

Method	Purpose
update()	INSERT, UPDATE, DELETE
query()	Retrieve multiple rows
queryForObject()	Retrieve a single object or scalar value
queryForList()	Retrieve rows as maps or simple values
batchUpdate()	Execute multiple updates efficiently
execute()	Execute SQL or JDBC callback operations

DataSource

DataSource provides database connections to Spring JDBC. Spring Boot commonly configures a HikariCP connection pool when JDBC dependencies are present.

RowMapper

RowMapper converts each row of a database ResultSet into a Java object.

DataAccessException

Spring JDBC translates SQL exceptions into unchecked exceptions belonging to the DataAccessException hierarchy.

5. Create a Spring Boot JDBC Project

Group: com.example

Artifact: spring-jdbc

Add the following dependencies:

- Spring Web
- JDBC API (Spring JDBC)
- MySQL Driver

The relevant Maven dependencies are:

```
썬<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>

<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
```

쩍In Spring Boot 4, spring-boot-starter-jdbc is the relevant starter for Spring JDBC.

6. Create MySQL Database and Table

Execute the following SQL queries:

```
썬CREATE DATABASE student_db;

USE student_db;

CREATE TABLE students (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
```

```
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    course VARCHAR(100) NOT NULL
);
```

Insert some sample records:

```
INSERT INTO students (name, email, course)
VALUES
('Rahul', 'rahul@gmail.com', 'Java'),
('Priya', 'priya@gmail.com', 'Python');
```

7. Configure Database Connection

Create or update src/main/resources/application.properties:

```
spring.application.name=student-jdbc

server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/student_db
spring.datasource.username=root
spring.datasource.password=${DB_PASSWORD}

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Set DB_PASSWORD as an environment variable before running the application.

Spring Boot automatically creates a DataSource and configures a JdbcTemplate bean when the required dependencies and database settings are available.

8. Project Folder Structure

```
student-jdbc
|
├── src/main/java/com/example/studentjdbc
|   |
```

```
│   ├── StudentJdbcApplication.java
│   │
│   ├── controller
│   │   └── StudentController.java
│   │
│   ├── service
│   │   └── StudentService.java
│   │
│   ├── repository
│   │   └── StudentRepository.java
│   │
│   └── model
│       └── Student.java
│
└── src/main/resources
    └── application.properties
```

9. Create the Student Model

File: model/Student.java

9. Create the Student Model

File: model/Student.java

```
package com.example.studentjdbc.model;

public class Student {

    private Long id;
    private String name;
    private String email;
    private String course;
```

```
    public Student() {
    }

    public Student(Long id, String name,
                   String email, String course) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.course = course;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}
```

썩This is a simple Java POJO representing a record in the students table.

Unlike JPA, Spring JDBC does not require annotations such as @Entity, @Table, or @Id.

10. Create the Repository Layer

File: repository/StudentRepository.java

This is the most important class because it performs all database operations.

썬package com.example.studentjdbc.repository;

```
import com.example.studentjdbc.model.Student;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public class StudentRepository {

    private final JdbcTemplate jdbcTemplate;

    public StudentRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    private final RowMapper<Student> studentRowMapper =
            (rs, rowNum) -> new Student(
                    rs.getLong("id"),
                    rs.getString("name"),
                    rs.getString("email"),
                    rs.getString("course")
            );

    // CREATE
    public int save(Student student) {

        String sql = """
                INSERT INTO students (name, email, course)
```

```
            VALUES (?, ?, ?)
            """;

    return jdbcTemplate.update(
            sql,
            student.getName(),
            student.getEmail(),
            student.getCourse()
    );
}

// READ ALL
public List<Student> findAll() {

    String sql = "SELECT * FROM students";

    return jdbcTemplate.query(sql, studentRowMapper);
}

// READ BY ID
public Optional<Student> findById(Long id) {

    String sql = "SELECT * FROM students WHERE id = ?";

    List<Student> students =
            jdbcTemplate.query(sql, studentRowMapper, id);

    return students.stream().findFirst();
}

// UPDATE
public int update(Long id, Student student) {

    String sql = """
            UPDATE students
            SET name = ?, email = ?, course = ?
            WHERE id = ?
            """;

    return jdbcTemplate.update(
            sql,
            student.getName(),
            student.getEmail(),
            student.getCourse(),
```

```
            id
        );
    }

    // DELETE
    public int deleteById(Long id) {

        String sql = "DELETE FROM students WHERE id = ?";

        return jdbcTemplate.update(sql, id);
    }
}
```

쩍11. Understanding Each CRUD Operation

A. INSERT – jdbcTemplate.update()

```
쩐String sql = "
    INSERT INTO students (name, email, course)
    VALUES (?, ?, ?)";

// jdbcTemplate.update(String sql,String... args);

int rows = jdbcTemplate.update(
    sql, "Arjun", "arjun@gmail.com", "Java"
);
```

- 쩍

  ? represents a SQL parameter placeholder.
- JdbcTemplate binds values using JDBC prepared-statement mechanisms.
- The method returns the number of affected rows.
- Normally, one successful insertion returns 1.
- The database automatically generates the student ID.

B. SELECT ALL – jdbcTemplate.query()

쩐String sql = "SELECT * FROM students";

```
List<Student> students =
        jdbcTemplate.query(sql, studentRowMapper);
```

쩍The query() method executes a SELECT statement and uses RowMapper to convert each row into a Student object.

C. SELECT BY ID

쩐String sql = "SELECT * FROM students WHERE id = ?";

```
List<Student> students =
        jdbcTemplate.query(sql, studentRowMapper, 1L);
```

쩍This retrieves students whose ID matches the supplied parameter.

Using query() with a list also makes it easy to handle the case where no matching record exists.

D. UPDATE

쩐String sql = """

```
    UPDATE students
    SET course = ?
    WHERE id = ?
    """;
```

```
int rows = jdbcTemplate.update(sql, "Spring Boot", 1L);
```

쩍The UPDATE statement modifies an existing record. The returned count indicates how many rows were affected.

E. DELETE

쩐String sql = "DELETE FROM students WHERE id = ?";

```
int rows = jdbcTemplate.update(sql, 1L);
```

쩍If the ID exists, the row is deleted and the affected-row count is normally 1. If it does not exist, the count is 0.

12. Create the Service Layer

File: service/StudentService.java

```
썬package com.example.studentjdbc.service;

import com.example.studentjdbc.model.Student;
import com.example.studentjdbc.repository.StudentRepository;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class StudentService {

    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    public void saveStudent(Student student) {
        repository.save(student);
    }

    public List<Student> getAllStudents() {
        return repository.findAll();
    }

    public Optional<Student> getStudentById(Long id) {
        return repository.findById(id);
    }

    public boolean updateStudent(Long id, Student student) {
        return repository.update(id, student) > 0;
    }

    public boolean deleteStudent(Long id) {
        return repository.deleteById(id) > 0;
    }
}
```

썩The service layer separates business logic from SQL database operations.

13. Create the REST Controller

File: controller/StudentController.java

썬package com.example.studentjdbc.controller;

```
import com.example.studentjdbc.model.Student;
import com.example.studentjdbc.service.StudentService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/students")
public class StudentController {

    private final StudentService service;

    public StudentController(StudentService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<String> save(@RequestBody Student student) {

        service.saveStudent(student);

        return ResponseEntity.status(HttpStatus.CREATED)
                .body("Student saved successfully");
    }

    @GetMapping
    public List<Student> getAll() {
        return service.getAllStudents();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getById(@PathVariable Long id) {
```

```
        return service.getStudentById(id)
                .map(ResponseEntity::ok)
                .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PutMapping("/{id}")
    public ResponseEntity<String> update(
            @PathVariable Long id,
            @RequestBody Student student) {

        boolean updated = service.updateStudent(id, student);

        if (!updated) {
            return ResponseEntity.notFound().build();
        }

        return ResponseEntity.ok("Student updated successfully");
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {

        boolean deleted = service.deleteStudent(id);

        if (!deleted) {
            return ResponseEntity.notFound().build();
        }

        return ResponseEntity.noContent().build();
    }
}
```

This controller exposes five REST endpoints for CRUD operations.

14. Test CRUD Operations Using Postman

POST

Create Student

http://localhost:8080/api/students

Request body:

쩐{
  "name": "Arjun",
  "email": "arjun@gmail.com",
  "course": "Java"
}

쩍Expected status: 201 Created.

GET

Get All Students

http://localhost:8080/api/students

Example response:

쩐[
  {
    "id": 1,
    "name": "Arjun",
    "email": "arjun@gmail.com",
    "course": "Java"
  }
]
쩍GET

Get Student by ID

http://localhost:8080/api/students/1

Returns the student if found, otherwise HTTP 404.

PUT

Update Student

http://localhost:8080/api/students/1

Request body:

쩐{
  "name": "Arjun Kumar",

```
  "email": "arjun@gmail.com",
  "course": "Spring Boot"
}
```

썩DELETE

Delete Student

http://localhost:8080/api/students/1

Expected status: 204 No Content when deletion succeeds.

15. How RowMapper Works Internally

A RowMapper is responsible for converting one row of a SQL result into one Java object.

```
썬RowMapper<Student> mapper = (rs, rowNum) -> {

    Student student = new Student();

    student.setId(rs.getLong("id"));
    student.setName(rs.getString("name"));
    student.setEmail(rs.getString("email"));
    student.setCourse(rs.getString("course"));

    return student;
};
```

썩Here:

- rs represents the current row of the ResultSet.
- rowNum represents the zero-based row number.
- rs.getLong() retrieves a numeric column value.
- rs.getString() retrieves a string column value.
- JdbcTemplate calls the mapper for every row returned.

For example, if the SQL query returns 10 records, the mapper executes 10 times.

16. query() vs queryForObject()

query()	queryForObject()

Returns a collection or callback-processed result	Returns a single result
Suitable for multiple records	Suitable for exactly one result
Empty result returns an empty list when using RowMapper overload	No rows causes EmptyResultDataAccessException
Used for findAll()	Used for counts and single-record queries

Example using queryForObject():

```
썬String sql = "SELECT COUNT(*) FROM students";

Integer count = jdbcTemplate.queryForObject(
        sql,
        Integer.class
);

System.out.println("Total students: " + count);
```

썩17. Transaction Management in Spring JDBC

A transaction groups multiple database operations into a single logical unit of work.

For example, when transferring money between two bank accounts:

1. Deduct ₹1,000 from Account A.
2. Add ₹1,000 to Account B.
3. If either operation fails, both changes should be rolled back.

Spring supports declarative transaction management through @Transactional.

```
썬@Service
```

```
public class BankService {

    private final JdbcTemplate jdbcTemplate;

    public BankService(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    @Transactional
    public void transfer(Long from, Long to, int amount) {

        jdbcTemplate.update(
            "UPDATE accounts SET balance = balance - ? WHERE id = ?",
            amount, from
        );

        jdbcTemplate.update(
            "UPDATE accounts SET balance = balance + ? WHERE id = ?",
            amount, to
        );
    }
}
```

썩This demonstrates transaction grouping, but a real banking application must additionally validate account existence, available balance, affected-row counts, concurrency, and monetary precision.

With an appropriate transaction manager and transactional database tables, Spring commits the operations together or rolls them back when an applicable exception occurs.

18. Advantages of Spring JDBC

1. Reduces repetitive JDBC boilerplate code.
2. Simplifies SQL query execution.
3. Automatically manages JDBC resources.
4. Supports parameterized SQL queries.
5. Integrates with Spring transaction management.
6. Provides consistent database exception handling.
7. Supports batch operations for improved efficiency.
8. Gives developers full control over SQL statements.
9. Does not require entity mapping annotations.
10. Works well for applications that need direct SQL control.

19. Limitations of Spring JDBC

1. SQL queries must generally be written manually.
2. Developers must map database results into Java objects.
3. There is no automatic entity relationship management like JPA.
4. Complex object relationships require additional SQL and mapping logic.
5. Database-specific SQL may reduce portability.
6. Schema changes may require manual query updates.
7. It does not provide Hibernate-style automatic dirty checking.
8. Pagination and dynamic queries require additional implementation.

Key concept to remember: Spring JDBC does not eliminate JDBC. It simplifies JDBC programming by allowing developers to focus on SQL and application logic while JdbcTemplate handles much of the connection, statement, result-processing, and exception-handling boilerplate.